A AgriProvider

DEVELOPER REFERENCE

API Documentation

Endpoint reference for authentication, provider operations, attendance, evidence review, assessor assignments, and the learner portal.

Version 1 · 10 November 2026

Scope

This document describes the Express API implemented in this repository. It does not describe planned integrations or imply official accreditation, certification, QA, funding, or payment behavior.

Base URL for local development: `http://localhost:5000`

Contents

1. Overview and conventions
2. Authentication and authorization
3. Errors and status codes
4. Health
5. Accounts and profile
6. Programmes and learners
7. Enrolments, sessions, attendance
8. Evidence requirements and submissions
9. Assessor review
10. Administrator assignments
11. Learner portal
12. Configuration and limits
13. Out-of-scope capabilities

Overview

The API is a JSON REST service backed by PostgreSQL. All routes except `GET /health` and public authentication routes require authentication. Resource ownership and role access are enforced on the server; clients must not treat hidden UI controls as authorization.

Unless a route says otherwise, JSON requests use `Content-Type: application/json` and successful responses use JSON. IDs are UUIDs. Timestamps are ISO 8601 values. JSON request bodies are limited to 16 KB; evidence uploads use multipart form data and are limited to 5 MB.

Local quick start

```
cd server
npm install
# Configure server/.env using server/.env.example
npm run migrate
npm start
# Check: http://localhost:5000/health
```

> **Database caution:** confirm the target database privately before applying migrations. Do not run development migrations against production without explicit approval.

Authentication

Supported roles are `provider` `learner` `assessor` `qa` `admin` . The API reads the user's role and active status from the database for each authenticated request.

Native clients: bearer token

Successful native login returns a one-hour JWT. Send it on protected requests:

```
Authorization: Bearer <accessToken>
```

Web clients: HTTP-only cookie

Send `X-Auth-Mode: cookie` and the exact configured `Origin` to login. Include credentials in fetch. The API sets an HTTP-only, SameSite=Strict cookie scoped to `/api`; the token is not returned in JSON.

```
POST /api/auth/login
Origin: http://localhost:8081
X-Auth-Mode: cookie
Content-Type: application/json

{"email":"person@example.com","password":"<password>"}
```

Browser clients must use `credentials: "include"`. Production cookies are Secure and production `WEB_ORIGIN` must use HTTPS.

Method and path	Access	Request / response
POST `/api/auth/register`	Public; rate limited	Body: `{fullName, email, password}`. Creates a provider account only; any client-supplied role is ignored. Returns `201 {user, accessToken}` for native clients or sets the web cookie and returns `201 {user}`. Duplicate email: `409`.
POST `/api/auth/login`	Public; rate limited	Body: `{email, password, accountType?}`. Optional accountType must match the existing role; it never changes the role. Returns the same native/web auth result as registration. Invalid credentials: `401`.
GET `/api/auth/me`	Authenticated	Returns `{user: {id, fullName, email, role}}`.
POST `/api/auth/logout`	Web cookie session	Clears the session cookie and returns `204`.

> **Privileged provisioning:** public registration cannot create learner, assessor, QA, or admin accounts. Admin, assessor, and QA users are provisioned from a trusted server terminal. Learner access requires a learner-role user linked to a learner profile.

Example protected request

```
curl http://localhost:5000/api/programmes \
  -H "Authorization: Bearer <accessToken>" \
```

```
  -H "Accept: application/json"
```

Errors and status codes

Errors use a JSON object with an `error` string. Some conflict responses also include a machine-readable `code`.

Status	Meaning	Typical cases
`400`	Invalid request	Malformed JSON, unsupported fields, invalid UUID/query/body, or invalid upload.
`401`	Authentication required / rejected	Missing, invalid, expired token; inactive account; incorrect login credentials.
`403`	Access denied	Wrong role or disallowed web-session origin.
`404`	Resource unavailable	Missing resource or resource outside the caller's permitted scope.
`409`	Conflict	Duplicate learner warning, duplicate enrolment, duplicate upload, stale review, or invalid workflow state.
`413`	Request too large	JSON body or evidence file exceeds its limit.
`429`	Too many requests	Authentication rate limit exceeded (10 attempts per 15 minutes).
`500`	Internal server error	Unexpected server failure; details are not returned.
`503`	Service unavailable	Database connection unavailable or configured file storage unavailable.

```
{"error":"Email or password is incorrect."}
```

Authentication failures intentionally do not disclose whether an email address exists. Database diagnostics belong in protected server logs, not API responses.

Health

Method and path	Access	Response
GET `/health`	Public	`200 {status:"ok", database:"connected"}`; returns `503 {status:"unavailable", database:"disconnected"}` if the database check fails.

Provider resources

Provider routes derive provider ownership from the authenticated user. Request bodies do not accept a provider ID.

Programmes · provider role

Method and path	Request / result
GET `/api/programmes`	List active programmes owned by the provider: `{programmes:[...]}`.
POST `/api/programmes`	Body: `{name, description?}`. Name 1–160 chars; description string/null up to 2,000 chars. Returns `201 {programme}`.
GET `/api/programmes/:id`	Get an owned active programme: `{programme}`.
PATCH `/api/programmes/:id`	Body contains at least one of `name`, `description`. Returns `{programme}`.
DELETE `/api/programmes/:id`	Archives the programme; returns `{message}`. Related history is preserved.

Learners · provider role

Method and path	Request / result
GET `/api/learners`	Query: `q` (prefix search, max 100 chars), `status` active|archived|all (default active), `limit` 1–100 (default 50), `offset` ≥0 (default 0). Returns `{learners,total,limit,offset}`.
POST `/api/learners`	Body: `{fullName, phone?, email?, confirmPossibleDuplicate?}`. Full name 1–120 chars; optional phone ≤32 chars and email ≤254 chars. Returns `201 {learner}`.
GET `/api/learners/:id`	Get one owned learner profile: `{learner}`.
PATCH `/api/learners/:id`	Partial update using `fullName`, `phone`, `email`; duplicate warning may require `confirmPossibleDuplicate:true`. Returns `{learner}`.
DELETE `/api/learners/:id`	Archives the learner and withdraws active enrolments; returns `{message}`.

Potential duplicate matches return `409` with code `POSSIBLE_DUPLICATE`. The provider must review and explicitly confirm if continuing.

Enrolments and attendance

Enrolments · provider role

Method and path	Request / result
GET `/api/enrolments`	Query: `learnerId?`, `programmeId?`, `status` active|withdrawn|all (default active), `limit` 1–100 (default 50), `offset` ≥0. Returns `{enrolments,total,limit,offset}`.
POST `/api/enrolments`	Body: `{learnerId, programmeId}` (UUIDs). Creates same-provider active enrolment. Returns `201 {enrolment}`; duplicate active pair returns `409`.
GET `/api/enrolments/:id`	Retrieve an owned enrolment: `{enrolment}`.
PATCH `/api/enrolments/:id`	Body: `{status:"active"}` or `{status:"withdrawn"}`. Returns `{enrolment}`.

Training sessions and attendance · provider role

Method and path	Request / result
GET `/api/sessions`	Optional query `programmeId`. Lists active sessions and counts.
POST `/api/sessions`	Body: `{programmeId,title,startsAt}`. Start time must be timezone-bearing ISO 8601. Returns `201 {session}`.
GET `/api/sessions/:id`	Retrieve an owned session: `{session}`.
PATCH `/api/sessions/:id`	Partial body: `{title?, startsAt?}`. Returns `{session}`.
DELETE `/api/sessions/:id`	Archives the session; returns `{message}`.
GET `/api/sessions/:id/attendance`	Returns `{session, records, summary}`. Summary counts present, absent, excused, and unrecorded active enrolments.
PUT `/api/sessions/:id/attendance`	Body: `{records:[{enrolmentId,status}]}`, 1–100 records. Status: present|absent|excused. Each enrolment may appear once. Atomic upsert; returns `{records}`.

Evidence

Evidence requirements · provider role

Method and path	Request / result
GET `/api/programmes/:programmeId/evidence-requirements`	Lists active checklist requirements: `{requirements}`.
POST `/api/programmes/:programmeId/evidence-requirements`	Body: `{title, guidance?}`. Title 1–160 chars; guidance ≤2,000 chars. Returns `201 {requirement}`.
GET `/api/programmes/:programmeId/evidence-requirements/:id`	Retrieve a requirement for an owned programme: `{requirement}`.
PATCH `/api/programmes/:programmeId/evidence-requirements/:id`	Partial body: `{title?, guidance?}`. Returns `{requirement}`.
DELETE `/api/programmes/:programmeId/evidence-requirements/:id`	Archives the requirement; returns `{message}`.

Evidence submissions · provider role

Method and path	Request / result
POST `/api/enrolments/:enrolmentId/evidence-requirements/:requirementId/submissions`	Multipart form with one field named `file`. Accepts PDF/JPEG/PNG up to 5 MB. Returns `201 {submission}`; upload starts as SUBMITTED.
GET `/api/enrolments/:enrolmentId/evidence-submissions`	Lists configured requirements with latest submission/review and completion counts: `{requirements,completion}`.
GET `/api/evidence-submissions/:submissionId`	Retrieve owned submission metadata: `{submission}`.
GET `/api/evidence-submissions/:submissionId/history`	Returns all versions and associated reviews: `{submissions,reviews}`.
GET `/api/evidence-submissions/:submissionId/file`	Downloads the private authorized file as an attachment; response is not JSON.
DELETE `/api/evidence-submissions/:submissionId`	Deletes only the latest unreviewed SUBMITTED version. Reviewed/approved and prior versions cannot be deleted.

File type is verified from content bytes, MIME type, and extension. The server stores private metadata and uses its configured storage adapter for file bytes. A revised version is allowed only after corrections are requested. Duplicate latest file returns `409 DUPLICATE_SUBMISSION`.

Evidence states: **NOT_SUBMITTED** **SUBMITTED** **UNDER_REVIEW** **APPROVED** **CORRECTION_REQUIRED** .

Assessor review

All routes require an assessor account and active assignment to the relevant programme. File access is authorized and private.

Method and path	Request / result
GET `/api/assessor/programmes`	Assigned programmes with learner and latest-evidence counts: `{programmes}`.
GET `/api/assessor/programmes/:programmeId/learners`	Active enrolled learners in an assigned programme: `{learners}`.
GET `/api/assessor/programmes/:programmeId/submissions`	Optional `status` query: all|SUBMITTED|APPROVED|CORRECTION_REQUIRED|UNDER_REVIEW. Returns latest submission per requirement.
GET `/api/assessor/submissions/:submissionId`	Retrieve assigned submission metadata and learner/programme/requirement details: `{submission}`.
GET `/api/assessor/submissions/:submissionId/file`	Download the authorized private evidence file.
GET `/api/assessor/submissions/:submissionId/history`	Returns all versions for the assigned evidence requirement: `{submissions}`.
GET `/api/assessor/submissions/:submissionId/reviews`	Returns version-bound review history: `{reviews}`.
POST `/api/assessor/submissions/:submissionId/reviews`	Body: `{decision, feedback?}`; decision APPROVED or CORRECTION_REQUESTED. Correction feedback is required, max 2,000 chars. Only the latest SUBMITTED version can be reviewed. Returns `201 {review}`.

> **Review behavior:** decisions are immutable and tied to the exact submission version and assignment. Approval sets status APPROVED; a correction request sets CORRECTION_REQUIRED.

Administrator assignments

These routes require an active administrator account. Administrator accounts are created through the trusted server CLI, never public registration.

Method and path	Request / result
GET `/api/admin/assessors`	List active assessor accounts: `{assessors}`.
GET `/api/admin/programmes`	List active programmes and provider names: `{programmes}`.
GET `/api/admin/assessor-assignments`	List active and historical assignments: `{assignments}`.
POST `/api/admin/assessor-assignments`	Body: `{programmeId, assessorUserId}` (UUIDs). Returns `201 {assignment}`. Existing active pair: `409 ACTIVE_ASSIGNMENT_EXISTS`.
PATCH `/api/admin/assessor-assignments/:assignmentId`	Body is one of `{status:"ACTIVE"}`, `{status:"INACTIVE"}`, or `{assessorUserId}` to reassign while preserving history. Returns `{assignment}`.

Learner portal

All routes require an authenticated learner role. The API finds the caller's linked learner profile; clients cannot choose an arbitrary learner ID.

Method and path	Request / result
GET `/api/learner/dashboard`	Returns the learner profile, programme enrolments, attendance summary and recent records, evidence requirements and all submission versions, current-version assessor feedback, evidence summary, and recent activity.
GET `/api/learner/programmes`	Returns the caller's linked programme enrolments: `{programmes}`.
GET `/api/learner/programmes/:id`	Returns one programme linked to the caller: `{programme}`. Unlinked programme returns 404.

Dashboard response shape

```
{
  "learner": { "id": "<uuid>", "learnerReference": "<reference>", "fullName": "<name>", "email": null, "phone": null },
  "programmes": [],
  "attendance": { "summary": { "recorded": 0, "present": 0, "absent": 0, "excused": 0 }, "recent": [] },
  "evidence": [],
  "evidenceSummary": { "total": 0, "notSubmitted": 0, "awaitingAssessment": 0, "approved": 0, "correctionsRequired": 0 },
  "recentActivity": []
}
```

Attendance is read-only for learners. No learner upload, notification, announcement, QA verification, or official completion endpoint is implemented.

Configuration and limits

Setting	Purpose
`DATABASE_URL`	PostgreSQL connection string; server-only secret.
`JWT_SECRET`	Signing secret, minimum 32 characters; server-only secret.
`WEB_ORIGIN`	Exact allowed web origin, e.g. `http://localhost:8081` locally; HTTPS required in production.
`EVIDENCE_STORAGE_DIR`	Optional local development evidence directory. Relative paths resolve from `server/`.

- Keep all secrets in `server/.env`; never place them in mobile config, source control, or this document.
- Authentication endpoints are limited to 10 attempts per 15-minute window.
- Passwords are 12–72 UTF-8 bytes and stored as bcrypt hashes.
- Evidence accepts PDF, JPEG, and PNG, up to 5 MB per file.
- Local filesystem evidence storage is for development only; production requires a configured private storage adapter.
- Most JSON request bodies are limited to 16 KB.

Not implemented

The API does not provide external AgriSETA/government integrations, payment execution, official accreditation or certification issuance, milestone eligibility, QA verification, learner evidence uploads, learner notifications or announcements, or a full audit system. Do not treat dashboard summaries as official completion or payment decisions.

> **Source of truth**
>
> This reference was compiled from the API routes, validation logic, and server documentation in the AgriProvider repository. The automated API suite uses an in-memory database stub; it is not proof of connectivity to a production database.